Case Study: DC Motor Position Control

From a physical motor model to a controller that holds a shaft angle.

A permanent-magnet DC motor is the workhorse actuator of control systems. Here we model one from first principles and design a controller that commands its **shaft position**. You will:

- build the motor transfer function from its electrical and mechanical parameters,
- see that the open loop (an integrator) cannot hold a position, and
- design and compare a **PD** and a **PID** position controller (before vs. after), checking the tracking with `stepinfo`.

Ties together `Mathematical Models/ElectricalSystems.m` and `PID Controllers/Intro.m`.

Run with `publish('DCMotorPositionControl.m')`, or step through with **Ctrl+Enter**.

Contents

Physical model

Electrical: $L\dot{i} + Ri = V - K_e\dot{\theta}$. Mechanical: $J\ddot{\theta} + b\dot{\theta} = K_t i$. With $K_t = K_e = K$, the transfer function from armature voltage V to shaft **angle** θ is

$$\frac{\Theta(s)}{V(s)} = \frac{K}{s\big[(Js+b)(Ls+R) + K^2\big]}.$$

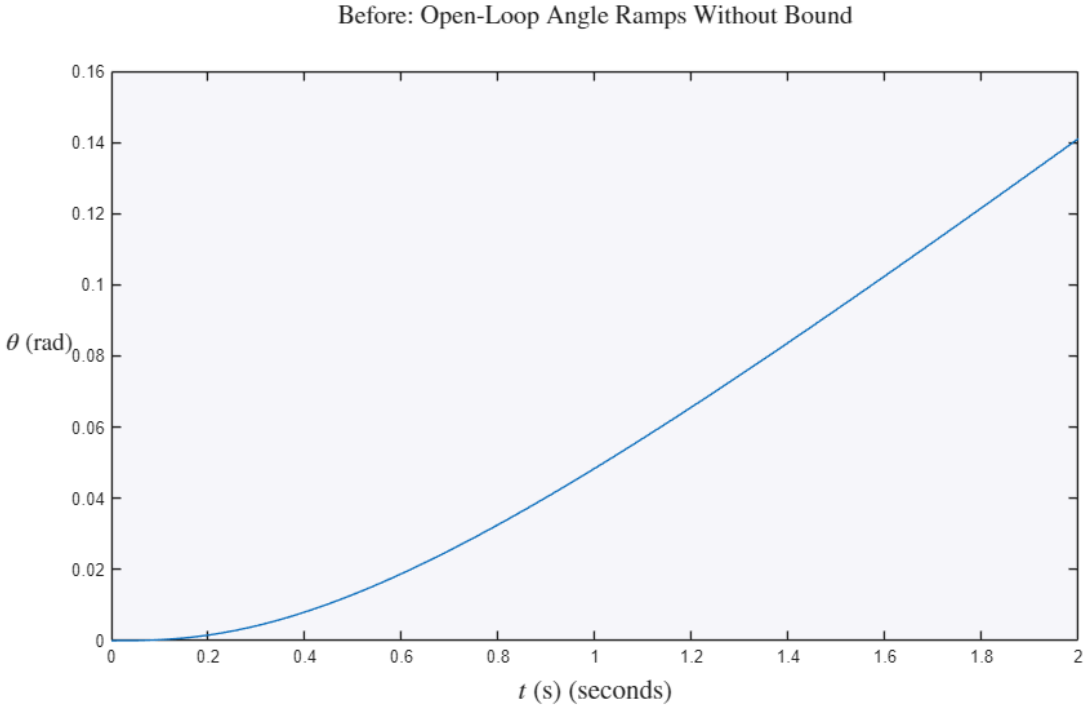
```
J = 0.01;    % rotor inertia (kg m^2)
b = 0.1;     % viscous friction (N m s)
K = 0.01;    % torque/back-emf constant
R = 1;       % armature resistance (ohm)
L = 0.5;     % armature inductance (H)

P = tf(K, conv([J b],[L R]) + [0 0 K^2]);   % voltage -> angular velocity
P = P * tf(1,[1 0]);                        % integrate velocity -> position
fprintf('Plant (voltage -> position):\n'); P  %#ok<NOPTS>
```

```
Plant (voltage -> position):

P =

              0.01
  -----------------------------
  0.005 s^3 + 0.06 s^2 + 0.1001 s

Continuous-time transfer function.
```

Before: open loop cannot hold a position

The plant has a free integrator (the $1/s$ from velocity to angle), so a constant voltage makes the shaft spin forever – its step response ramps without ever settling at a target angle.

```
figure
step(P, 0:0.01:2)
title('Before: Open-Loop Angle Ramps Without Bound','Interpreter','latex','FontSize',15)
ylabel('$\theta$ (rad)','Interpreter','latex','FontSize',14); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$ (s)','Interpreter','latex','FontSize',16)
```



What we do: a PD position controller

Position control needs derivative action for damping. Close a unity- feedback loop with $C(s) = K_p + K_d s$.

```
Kp = 20; Kd = 8;
Cpd = pid(Kp,0,Kd);
T_pd = feedback(Cpd*P, 1);
```

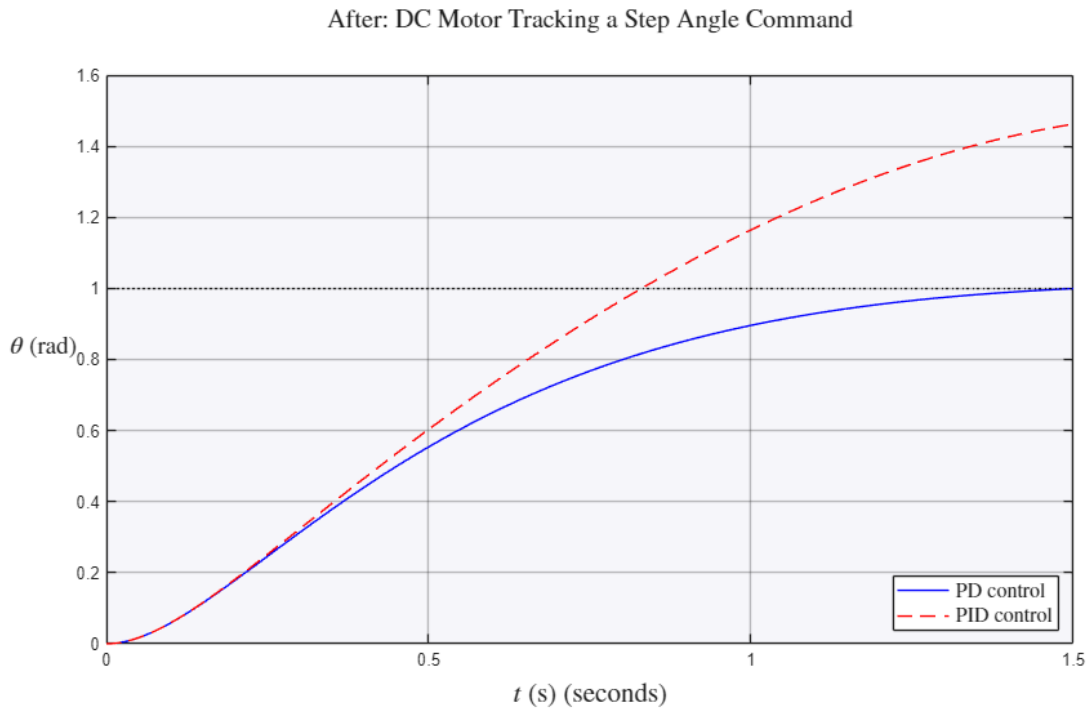
Adding integral action: PID

A PD loop can leave a small steady-state error against friction/torque disturbances; integral action removes it.

```
Ki = 30;
Cpid = pid(Kp,Ki,Kd);
T_pid = feedback(Cpid*P, 1);
```

After: tracking a commanded angle (before vs. after)

```
figure
step(T_pd,'b', T_pid,'r--', 0:0.001:1.5)
hold on; yline(1,'k:','HandleVisibility','off'); hold off
grid on
legend('PD control','PID control','Interpreter','latex','FontSize',12,'Location','southeast')
title('After: DC Motor Tracking a Step Angle Command','Interpreter','latex','FontSize',15)
ylabel('$\theta$ (rad)','Interpreter','latex','FontSize',14); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$ (s)','Interpreter','latex','FontSize',16)
```



Quantify what changed

`stepinfo` reports the transient and steady-state numbers behind the curves.

```
info_pd  = stepinfo(T_pd);
info_pid = stepinfo(T_pid);
fprintf('PD : overshoot %.1f%%, settling %.3f s, ss error %.4f\n', ...
    info_pd.Overshoot,  info_pd.SettlingTime,  1-dcgain(T_pd))
fprintf('PID: overshoot %.1f%%, settling %.3f s, ss error %.4f\n', ...
    info_pid.Overshoot, info_pid.SettlingTime, 1-dcgain(T_pid))
```

```
PD : overshoot 1.5%, settling 1.333 s, ss error 0.0000
PID: overshoot 50.0%, settling 13.075 s, ss error 0.0000
```

Summary

- The motor model is a 3rd-order plant with a free integrator: it spins, it does not hold a position on its own.
- Closing a position loop with PD adds the damping the integrator lacks; PID additionally zeroes the steady-state error.
- `stepinfo` and `dcgain` turn the "after" plot into hard numbers.

Try it: lower K_d and watch overshoot and ringing return – the same derivative-damping tradeoff seen in `PID Controllers/Intro.m`.

Try it yourself

- Remove the integral term (PD only) and confirm a small steady-state error creeps in against the plant dynamics.
- Double `Kp` and notice the response speed up but overshoot grow – the classic proportional tradeoff.